

Python Programming

Day 6: 상속과 클래스 변수

Inheritance, Method Overriding, super(), Class Variable

Contents

1	학습 목표	2
2	상속의 기본 개념	2
2.1	상속 문법	2
2.2	부모 메서드 사용하기	2
2.3	최종 코드: 01_inheritance.py	3
3	메서드 오버라이딩	3
3.1	메서드 탐색 순서	4
3.2	최종 코드: 02_method_overriding.py	4
4	자식 생성자와 super()	4
4.1	부모 생성자 호출 없이 작성한 경우	4
4.2	super()의 역할	5
4.3	최종 코드: 03_super.py	5
5	클래스 변수	6
5.1	클래스 변수 정의	6
5.2	클래스 이름으로 접근하기	6
5.3	여러 객체가 같은 클래스 변수 사용하기	7
5.4	최종 코드: 04_class_variable.py	7
6	클래스 변수 변경과 인스턴스 변수의 차이	7
6.1	객체를 통한 대입 주의하기	8
7	Day 6 종합 실습: 학교 구성원 관리	8
7.1	프로그램 요구사항	8
7.2	최종 코드: practice_day06.py	9
7.3	실행 결과	9
7.4	프로그램 구조 분석	10
8	실습 파일 목록	10
9	핵심 요약	10

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 상속을 사용하는 이유와 부모 클래스, 자식 클래스의 관계 설명하기
- 자식 클래스에서 부모 클래스의 속성과 메서드 사용하기
- 자식 클래스에서 부모 메서드를 재정의하는 오버라이딩 이해하기
- `super()`를 이용하여 부모 클래스의 생성자 호출하기
- 부모 속성과 자식 고유 속성을 하나의 객체에 함께 저장하기
- 클래스 변수와 인스턴스 변수의 차이 설명하기
- 클래스 변수를 클래스 이름으로 읽고 변경하기
- 상속과 클래스 변수를 결합한 간단한 프로그램 구현하기

2. 상속의 기본 개념

상속(inheritance)은 기존 클래스의 속성과 메서드를 새로운 클래스가 이어받아 사용하는 기능이다. 공통 기능을 부모 클래스에 한 번만 작성하고, 자식 클래스에서는 필요한 기능만 추가할 수 있다.

용어	의미
부모 클래스	공통 속성과 메서드를 제공하는 클래스
자식 클래스	부모 클래스를 상속받아 확장하는 클래스
상속	기존 클래스의 기능을 이어받는 관계

예를 들어 모든 학생은 사람이다. 따라서 사람에게 공통으로 필요한 이름은 `Person` 클래스에 두고, 학생에게만 필요한 전공은 `Student` 클래스에 추가할 수 있다.

2.1. 상속 문법

자식 클래스 이름 뒤의 괄호 안에 부모 클래스 이름을 적는다.

```
1 class Parent:
2     pass
3
4
5 class Child(Parent):
6     pass
```

`Child`는 `Parent`를 상속받았으므로 부모 클래스에 정의된 기능을 사용할 수 있다.

2.2. 부모 메서드 사용하기

```
1 class Person:
2     def greet(self):
3         print("Hello")
4
5
6 class Student(Person):
7     pass
```

```

8
9
10 student = Student()
11 student.greet()

```

실행 결과는 다음과 같다.

Hello

Student 클래스 안에는 greet()가 직접 정의되어 있지 않다. 그러나 Person을 상속받았으므로 Student 객체에서도 부모 메서드를 호출할 수 있다.

2.3. 최종 코드: 01_inheritance.py

```

1 class Person:
2     def greet(self):
3         print("Hello")
4
5
6 class Student(Person):
7     pass
8
9
10 student = Student()
11 student.greet()

```

실습

Animal 클래스에 eat() 메서드를 만들고, 이를 상속받는 Dog 클래스를 작성한다. Dog 객체를 생성하여 부모 클래스의 eat()를 호출한다.

핵심 포인트

상속은 부모 객체를 자식 객체 안에 복사하는 기능이 아니다. 자식 클래스가 부모 클래스의 정의를 이용할 수 있도록 클래스 사이에 관계를 만드는 기능이다.

3. 메서드 오버라이딩

자식 클래스는 부모 클래스에서 물려받은 메서드를 그대로 사용할 수도 있고, 같은 이름으로 다시 정의하여 동작을 바꿀 수도 있다. 이를 메서드 오버라이딩(method overriding)이라고 한다.

```

1 class Person:
2     def introduce(self):
3         print("I am a person.")
4
5
6 class Student(Person):
7     def introduce(self):
8         print("I am a student.")

```

Student 객체에서 introduce()를 호출하면 자식 클래스에 정의된 메서드가 우선 사용된다.

```

1 student = Student()
2 student.introduce()

```

실행 결과는 다음과 같다.

I am a student.

3.1. 메서드 탐색 순서

객체에서 메서드를 호출하면 Python은 먼저 객체의 클래스에서 해당 메서드를 찾는다. 없으면 부모 클래스로 올라가며 찾는다.

```
Student object
-> Student class
-> Person class
-> object class
```

자식 클래스에 같은 이름의 메서드가 있으면 부모 메서드보다 먼저 발견되므로 자식 메서드가 실행된다.

3.2. 최종 코드: 02_method_overriding.py

```
1 class Person:
2     def introduce(self):
3         print("I am a person.")
4
5
6 class Student(Person):
7     def introduce(self):
8         print("I am a student.")
9
10
11 person = Person()
12 student = Student()
13
14 person.introduce()
15 student.introduce()
```

실행 결과는 다음과 같다.

```
I am a person.
I am a student.
```

실습

Animal 클래스에 `sound()` 메서드를 만들고 Dog 클래스에서 같은 메서드를 재정의한다. 부모 객체와 자식 객체에서 각각 메서드를 호출하여 결과를 비교한다.

핵심 포인트

오버라이딩은 부모 클래스의 코드를 직접 수정하지 않고 자식 클래스에 맞는 동작을 정의하는 방법이다.

4. 자식 생성자와 `super()`

자식 클래스가 자신만의 생성자를 정의하면 그 생성자가 객체 생성 직후 실행된다. 이때 부모 클래스의 생성자는 자동으로 실행되지 않으므로, 부모가 담당하는 초기화가 필요하다면 직접 호출해야 한다.

4.1. 부모 생성자 호출 없이 작성한 경우

```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5
6 class Student(Person):
7     def __init__(self, name, major):
8         self.major = major

```

이 코드에서 Student 객체에는 major만 저장되고, 부모 생성자의 `self.name = name`은 실행되지 않는다.

4.2. super()의 역할

`super()`는 현재 자식 클래스의 부모 클래스를 기준으로 메서드에 접근한다.

```

1 super().__init__(name)

```

이 문장은 부모 클래스의 `__init__()`를 호출하여 이름을 초기화한다.

```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5
6 class Student(Person):
7     def __init__(self, name, major):
8         super().__init__(name)
9         self.major = major

```

객체 생성 과정은 다음과 같다.

1. `Student("Alice", "Physics")`를 호출한다.
2. `Student.__init__()`가 실행된다.
3. `super().__init__(name)`이 부모 생성자를 호출한다.
4. 부모 생성자가 `self.name`을 만든다.
5. 자식 생성자가 `self.major`를 만든다.

결과적으로 하나의 Student 객체 안에 부모가 초기화한 `name`과 자식이 초기화한 `major`가 함께 저장된다.

```

student
  name -> "Alice"
  major -> "Physics"

```

4.3. 최종 코드: 03_super.py

```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5
6 class Student(Person):
7     def __init__(self, name, major):
8         super().__init__(name)

```

```

9         self.major = major
10
11     def show_info(self):
12         print(f"Name : {self.name}")
13         print(f"Major : {self.major}")
14
15
16 student = Student("Alice", "Physics")
17 student.show_info()

```

실행 결과는 다음과 같다.

```

Name : Alice
Major : Physics

```

실습

Vehicle 클래스가 brand를 저장하도록 만들고, 이를 상속받는 Car 클래스가 year를 추가로 저장하도록 작성한다. 자식 생성자에서 super()를 사용하고 두 속성을 모두 출력한다.

핵심 포인트

super().__init__()는 새로운 부모 객체를 만드는 것이 아니다. 지금 생성 중인 동일한 자식 객체를 부모 클래스의 생성자 로직으로 초기화한다.

5. 클래스 변수

인스턴스 변수는 객체마다 따로 저장되는 값이다. 반면 클래스 변수(class variable)는 클래스에 속하며 같은 클래스로 만든 모든 객체가 공유하는 값이다.

5.1. 클래스 변수 정의

클래스 변수는 메서드 밖, 클래스 블록 안에 작성한다.

```

1 class Book:
2     category = "Programming"
3
4     def __init__(self, title):
5         self.title = title

```

여기서 category는 클래스 변수이고 self.title은 인스턴스 변수이다.

구분	정의 위치	저장 방식
클래스 변수	클래스 안, 메서드 밖	클래스가 하나를 공유
인스턴스 변수	일반적으로 __init__() 안	객체마다 별도로 저장

5.2. 클래스 이름으로 접근하기

클래스 변수는 클래스 이름으로 접근하는 것이 가장 명확하다.

```

1 print(Book.category)

```

객체를 통해서도 읽을 수 있지만, 공유 데이터라는 사실을 분명히 나타내기 위해 클래스 이름을 사용하는 편이 좋다.

```

1 print(book1.category)

```

5.3. 여러 객체가 같은 클래스 변수 사용하기

```
1 book1 = Book("Python Basics")
2 book2 = Book("C++ Guide")
```

각 객체의 title은 다르지만 category는 같은 클래스 값을 사용한다.

```
Book class
    category -> "Programming"
```

```
book1
    title -> "Python Basics"
```

```
book2
    title -> "C++ Guide"
```

5.4. 최종 코드: 04_class_variable.py

```
1 class Book:
2     category = "Programming"
3
4     def __init__(self, title):
5         self.title = title
6
7     def show_info(self):
8         print(f"Title : {self.title}")
9         print(f"Category : {Book.category}")
10
11
12 book1 = Book("Python Basics")
13 book2 = Book("C++ Guide")
14
15 book1.show_info()
16 print()
17 book2.show_info()
```

실행 결과는 다음과 같다.

```
Title : Python Basics
Category : Programming
```

```
Title : C++ Guide
Category : Programming
```

실습

Student 클래스에 클래스 변수 school = "KU"를 만들고, 인스턴스 변수로 이름을 저장한다. 학생 객체 두 개에서 이름과 학교를 출력한다.

6. 클래스 변수 변경과 인스턴스 변수의 차이

클래스 이름을 통해 클래스 변수를 변경하면 이후 모든 객체가 변경된 값을 공유한다.

```
1 class Student:
2     school = "MIT"
3
```



```

4     def __init__(self, name):
5         self.name = name
6
7
8     student1 = Student("Alice")
9     student2 = Student("Bob")
10
11    print(Student.school)
12
13    Student.school = "KU"
14
15    print(student1.school)
16    print(student2.school)

```

실행 결과는 다음과 같다.

```

MIT
KU
KU

```

`Student.school = "KU"`는 클래스에 저장된 공유 값을 변경한다. 따라서 기존에 생성된 객체들도 객체 자체에 같은 이름의 별도 속성이 없다면 변경된 값을 읽는다.

6.1. 객체를 통한 대입 주의하기

다음 대입은 클래스 변수를 직접 바꾸는 것과 다르다.

```

1 student1.school = "Harvard"

```

이 코드는 일반적으로 `student1` 객체 안에 `school`이라는 새로운 인스턴스 변수를 만든다. 이후 `student1.school`은 인스턴스 값을 우선 읽고, 다른 객체는 여전히 클래스 변수를 읽는다.

```

Student class
  school -> "KU"

student1
  name   -> "Alice"
  school -> "Harvard"

student2
  name   -> "Bob"

```

핵심 포인트

공유 값을 변경할 때는 `ClassName.variable` 형식을 사용한다. 객체 이름으로 대입하면 같은 이름의 인스턴스 변수가 새로 생길 수 있다.

7. Day 6 종합 실습: 학교 구성원 관리

이번 종합 실습에서는 상속, `super()`, 메서드 오버라이딩, 클래스 변수를 함께 사용한다.

7.1. 프로그램 요구사항

1. `Person` 클래스에 클래스 변수 `school = "KU"`를 만든다.
2. `Person` 생성자는 `name`을 저장한다.

3. `Person.show_info()`는 이름과 학교를 출력한다.
4. `Student`는 `Person`을 상속받는다.
5. `Student` 생성자는 `super()`로 이름을 초기화하고 전공을 추가한다.
6. `Student.show_info()`를 오버라이딩하여 이름, 학교, 전공을 출력한다.
7. 학생 객체 두 개를 생성하여 정보가 올바르게 출력되는지 확인한다.

7.2. 최종 코드: practice_day06.py

```
1 class Person:
2     school = "KU"
3
4     def __init__(self, name):
5         self.name = name
6
7     def show_info(self):
8         print(f"Name : {self.name}")
9         print(f"School : {Person.school}")
10
11
12 class Student(Person):
13     def __init__(self, name, major):
14         super().__init__(name)
15         self.major = major
16
17     def show_info(self):
18         print(f"Name : {self.name}")
19         print(f"School : {Person.school}")
20         print(f"Major : {self.major}")
21
22
23 student1 = Student("Alice", "Physics")
24 student2 = Student("Bob", "Engineering")
25
26 student1.show_info()
27 print()
28 student2.show_info()
```

7.3. 실행 결과

```
Name : Alice
School : KU
Major : Physics
```

```
Name : Bob
School : KU
Major : Engineering
```

7.4. 프로그램 구조 분석

기능	사용한 개념
공통 이름 저장	부모 클래스 생성자
학생 전공 저장	자식 클래스 인스턴스 변수
부모 초기화 재사용	<code>super()</code>
학생 정보 출력 방식 변경	메서드 오버라이딩
학교 이름 공유	클래스 변수
서로 다른 학생 정보 저장	인스턴스 변수

핵심 포인트

상속은 공통 기능을 재사용하고, 오버라이딩은 자식 클래스에 맞게 동작을 바꾸며, 클래스 변수는 모든 객체가 공유해야 하는 값을 저장한다.

8. 실습 파일 목록

실습

Day 6의 학습 파일은 다음과 같이 관리한다.

- 01_inheritance.py
- 02_method_overriding.py
- 03_super.py
- 04_class_variable.py
- practice_day06.py
- python_day06.tex
- python_day06.pdf

9. 핵심 요약

- 상속은 기존 클래스의 속성과 메서드를 새로운 클래스가 이어받는 기능이다.
- 부모 클래스는 공통 기능을 제공하고 자식 클래스는 이를 재사용하거나 확장한다.
- `class Child(Parent):` 형식으로 상속 관계를 만든다.
- 자식 객체는 자식 클래스에 없는 메서드를 부모 클래스에서 찾아 사용할 수 있다.
- 자식 클래스가 부모와 같은 이름의 메서드를 정의하면 자식 메서드가 우선 실행된다.
- 같은 이름으로 부모 메서드의 동작을 바꾸는 것을 메서드 오버라이딩이라고 한다.
- 자식 클래스가 생성자를 직접 정의하면 부모 생성자는 자동으로 호출되지 않는다.
- `super().__init__()`로 부모 생성자의 초기화 코드를 재사용할 수 있다.
- 부모 생성자와 자식 생성자는 동일한 자식 객체의 속성을 함께 초기화한다.
- 클래스 변수는 클래스에 속하며 여러 객체가 하나의 값을 공유한다.

- 인스턴스 변수는 객체마다 독립적으로 저장된다.
- 공유 값을 읽거나 변경할 때는 `ClassName.variable` 형식을 사용하는 것이 명확하다.
- 객체를 통해 클래스 변수와 같은 이름에 대입하면 새로운 인스턴스 변수가 생길 수 있다.